

Extended Abstract

Motivation When given difficult problems, we, humans, tend to think for longer to improve our reliability in our decisions. The phrasing of the problem also makes a difference. Through this project, we wanted to explore if we can enable LLMs to effectively make use of additional computation at test-time to improve the accuracy of their responses while seeking ways to reduce inference latency with prompt structures.

Method In addition to implementing SFT and RLOO, we implemented best-of-N method, which samples n completions per prompt rather than one from fine-tuned model during test time inference. To experiment with prompt-engineering, we generate a sequence of prompt formats, where each prompt has less and less non-critical words and retrain the base model with the prompt that leads to the most promising results.

Implementation For all the experiments, we use Qwen 2.5 0.5B Base model and fine-tune using SFT on `Asap7772/cog_behav_all_strategies` Warmstart dataset. For SFT, we used the following training settings: batch size of 2 with gradient accumulation steps of 4 (effective batch size = 8), learning rate of $1e-5$, 7 training epochs. For RLOO, due to the out of memory issues, we were only able to train using $k=4$ for 1200 prompts with a batch size of 1. For best-of-N, we experimented with $n=1,2,4,8$ and 16 and generated 7 different prompts by progressively removing "non-essential" words from the prompt.

Results We achieved a score of 0.3355 using supervised fine-tuning (SFT), while the RLOO approach yielded a lower score of 0.2385, likely due to limitations imposed by out-of-memory constraints that restricted us to use smaller batch sizes during training. With the help of the extension part, we observed a score of 0.8420 using $n = 16$ and prompt format 4.

Discussion Throughout this paper, we show that in the context of the Countdown task, removing unnecessary tokens from the prompt can be an effective way to reduce compute time during inference and even improve performance. However, words in the default prompt were categorized into critical and non-critical manually, which will often be impractical. We believe that it would be interesting for future research to train a model to classify tokens in a prompt into critical vs non-critical and use that information. Best-of-N method demonstrates a significant improvement in performance by improving robustness. To extend this work, investigation into other methods, such as beam and lookahead search, can illuminate ways to scale test-time computation.

Conclusion In this work, we present a method to fine-tune a large language model, first with supervised fine tuning and then with RLOO, to improve performance on the Countdown task. We then demonstrate how prompt engineering and output sampling can be used to improve mathematical reasoning. In particular, we find that reducing the length of the prompt by removing non-critical tokens can both boost model performance and reduce compute costs during test-time inference. We believe this to be a novel and interesting result since it demonstrates that in the context of mathematical reasoning, decreasing prompt lengths by simply cutting filler tokens after a user has inputted a prompt may be able to both improve generation quality and decrease compute times, and since this phenomenon has been underexplored in prior literature.

Inference Remix: Prompt Engineering and Output Sampling with Fine-Tuned LLMs

Nomin-Erdene Bayarsaikhan

Department of Computer Science
Stanford University
nomin@stanford.edu

Ronny Junkins

Department of Computer Science
Stanford University
rjunkins@stanford.edu

Abstract

While Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of tasks, they continue to struggle with complex reasoning problems. To address this issue, we explore inference-time strategies on a supervised fine-tuned model, leveraging best-of-N sampling and prompt engineering. Our goal is to investigate the role of prompt structure and whether if we can enable LLMs to effectively make use of additional computation at test-time to improve the accuracy of their responses. Experimental results demonstrate that best-of-N sampling significantly enhance the performance on the Countdown task. Additionally, removing non-essential words from the prompt not only preserves performance but also leads to a 10% speedup in compute time.

1 Introduction

Large language models (LLMs) have made recent stunning improvements in their ability to perform complex quantitative reasoning and solve difficult math problems. Even so, many models still make mistakes and hallucinate wrong results. In addition, when presented with the same problem in different formats, LLMs arrive at different solutions. This inconsistency highlights a key limitation: the sensitivity of LLMs to prompt phrasing and input structure. Subtle changes in wording or formatting can lead to significant variations in output. To investigate this issue and the possibility of LLMs' test-time capability, our project explores two test time inference strategies: output sampling and prompt engineering.

When given difficult problems, we, humans, tend to think for longer to improve our reliability in our decisions. Through this project, we wanted to explore if we can enable LLMs to effectively make use of additional computation at test-time to improve the accuracy of their responses. Such a capability at test-time could unlock new avenues in agentic and reasoning tasks: for example, smaller LLM models could be deployed on-device in lieu of large, datacenter scale LLMs.

Prior work focusing on inference-time computation provides mixed results. While some of the work shows that LLMs can use test-time computation to improve their outputs (Bi et al., 2025; Snell et al., 2024), others demonstrate that the effectiveness of this method can fail for complex tasks, such as math reasoning (Stechly et al., 2023; Huang et al., 2024). These conflicting findings motivate the need to experiment with the incorporation of test-time inference by scaling test-time compute. To understand the potential benefits better specifically for the fine-tuned Qwen 2.5 0.5B Base model for the math reasoning Countdown task, we carry out experiments with best-of-N sampling: sampling N outputs in "parallel" from the fine-tuned model and selecting the one that scores the highest per a reward model.

Another effective way to extend upon the already fine-tuned model is to experiment with different prompt formats. Running experiment with prompt engineering doesn't require labeled datasets if

we already have a fine-tuned model, and it is less resource intensive. To provide a qualitative and quantitative analysis of fine-tuned Qwen 2.5 0.5B model, we specifically explore the question "Can removing non-essential tokens during inference-time lead to a better performance?".

2 Related Work

Best-of-N sampling. Cobbe et al. (2021) finds that even the largest transformer models fail to achieve a high test score despite the simplicity of the given math problem. Thus, they propose to generate many candidate solutions during test time by training a verifier. This approach improves accuracy by selecting the most logically consistent answer, even when the original model exhibits variability or hallucinations. This work highlights the benefits of test-time computation as their finding demonstrates that verification provides a significant performance boost relative to a finetuning baseline.

Snell et al. (2024) finds that best-of-N is more effective on easier questions and at higher compute-time budgets by conducting experiments to optimize process reward model (PRM) via three search methods: best-of-N weighted, beam search and lookahead search. The work concludes that on easy and medium questions, test-time compute can easily cover up for additional pretraining. Since the Countdown task can be categorized as simple enough, this article provides grounds for our experiments.

Prompt Engineering. Zhou et al. (2022) introduces the idea of *Algorithmic Prompting*, which involves providing a detailed description of the algorithm execution and using explicit explanation to remove ambiguity, which highlights the importance of structure of prompts. Wei et al. (2023) demonstrates that a chain of thought prompting, a series of intermediate reasoning steps, can significantly improve the ability of LLMs to perform complex reasoning. This work further emphasizes that prompt structure matters.

Inspired by these related work and existing methods, we hope to combine prompt engineering and output sampling during inference time in a way that leverages the strengths of both to maximize accuracy through verifier-guided selection while minimizing inference compute time through prompt compression.

3 Method

3.1 SFT

For the default part of the project, we first explored the implementation of Supervised Fine-Tuning (SFT) to fine-tune a pre-trained model Qwen 2.5 0.5B Base LLM on high-quality data from the Countdown task to obtain a reference model, π_{ref} . The following supervised learning objective is optimized over queries x and completions y that are drawn from an expert distribution:

$$\max_{\theta} \mathbb{E}_{x,y \in \mathcal{D}} \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t \mid x, y_{<t}).$$

3.2 RLOO

To extend the SFT method with an online RL algorithm, we implemented REINFORCE Leave One-Out (Pareja et al., 2024), which samples new responses from the current snapshot of the learning policy, score it under the reward model (compute score function from Gandhi et al. (2025)) and performs a policy gradient update on parameters. The objective function for RLOO is formally written as:

$$\frac{1}{k} \sum_{i=1}^k \left[R(y_{(i)}, x) - \frac{1}{k-1} \sum_{j \neq i} R(y_{(j)}, x) \right] \nabla \log \pi_{\theta}(y_{(i)} \mid x) \quad \text{for } y_{(1)}, \dots, y_{(k)} \stackrel{i.i.d.}{\sim} \pi_{\theta}(\cdot \mid x).$$

For the extension portion of this project, we apply two approaches during Test-Time inference to improve the performance of our model on the Countdown task. We describe each approach as follows, and in the results show how combining them improves the performance of the model by combining benefits from both approaches.

3.3 Best-of-N

During test time inference, rather than sampling one completion per prompt from our model, we sample n completions from the SFT model. We use the `compute_score` function from Gandhi et al. (2025) to score each of these outputs, and use the completion with the highest score as the final answer. This approach improves the robustness by offloading some decision-making to the reward function and helps mitigate generation randomness and model uncertainty. Samples are taken in batch using VLLM to improve sample generation times.

3.4 Prompt engineering

In addition, we explore prompt engineering during inference time. That is, during supervised fine-tuning, the Qwen 2.5 0.5B Base model is fine tuned using the following prompt:

```
A conversation between User and Assistant. The user asks a question, and the
Assistant solves it. The assistant first thinks about the reasoning process in
the mind and then provides the user with the answer. \n

User: Using the numbers {numbers}, create an equation that equals {target}. You
can use basic arithmetic operations (+, -, *, /) and each number can only be used
once. Show your work in <think></think> tags. And return the final answer in
<answer></answer> tags, for example <answer>(1 + 2) / 3 </answer>. \n
Assistant: Let me solve this step by step.
```

Here, `numbers` is replaced by a vector of integers such as `[95, 36, 32]`, and `target` is replaced with an integer such as `91`. This is consistent with the format used throughout `Asap7772/cog_behav_all_strategies`. We noticed that there were a significant number of "filler tokens", or words in the prompt that were not critical to expressing the problem objective to the model. Hence, our next step was to label each of the characters in the given prompt as critical vs non-critical. For reference, the following is the prompt with only critical tokens remaining:

```
conversation User and Assistant. user asks question, Assistant solves assistant
provides the answer. \n
User: Using {numbers}, create equation equals {target}. use (+, -, *,
/) and each number once. Show work in <think></think> return answer in
<answer></answer> tags. \n
Assistant: Let me solve this
```

We then generate a sequence of prompt formats, where each prompt removes more and more non-critical tokens. We do this until we reach a prompt which entirely consists of critical tokens, prompt format 7, and is identical to above. See Table 1 for specifics on the number of characters in each prompt format, and see Appendix A for a full list of all prompt formats used. Here, we also note that prompt format 3 is special since it removes the line `Show your work in <think> </think> tags`. In particular, this prompt format is used to test how well the model functions without critical tokens, which in this case means how well is it able to perform when not directed to show its work at every step.

We then fine-tune the model again with SFT with a prompt format that gives the most promising result.

Of note, we do the above labeling of critical tokens and generation of the different prompt formats by hand. Training a model to classify tokens in a given prompt as critical vs non-critical and using this information to automatically generate reduced prompt formats is an interesting area for future work.

	OG	F2	F3*	F4	F5	F6	F7
total chars	567	522	478	486	429	386	301
% of original	100	92.1	85.7	75.7	0.649	0.681	0.531

Table 1: Prompt formats are constructed by removing non-critical words from the original prompt. In no cases are characters added. Note that F3 is special as it removes critical <think> tags

4 Experimental Setup

In the experiments presented in the results section, we use the Qwen 2.5 0.5B Base model fine tuned on `Asap7772/cog_behav_all_strategies` and a dataset from Gandhi et al. (2025), `Jiayi-Pan/Countdown-Tasks-3to4` for RLOO. We test on the original countdown test prompts provided for the milestone. For sampling parameters, we use a temperature of 0.7, a TopP value of 1, and 1024 max tokens per VLLM generation. Finally, we test n samples per prompt, using $n \in \{1, 2, 4, 8, 16\}$ in all experiments.

5 Results

In this section, we present and analyze our primary results, which can be found in Table 2, Figure 3, and Figure 4.

5.1 Quantitative Evaluation

SFT We first fine-tuned the base model using SFT using the Warmstart dataset and obtained a score of 0.3355. As seen in Figure 1 (a) and (b), the training and validation loss curve go down over epochs, and the final evaluation score exceeds the minimum expected score of 0.3.

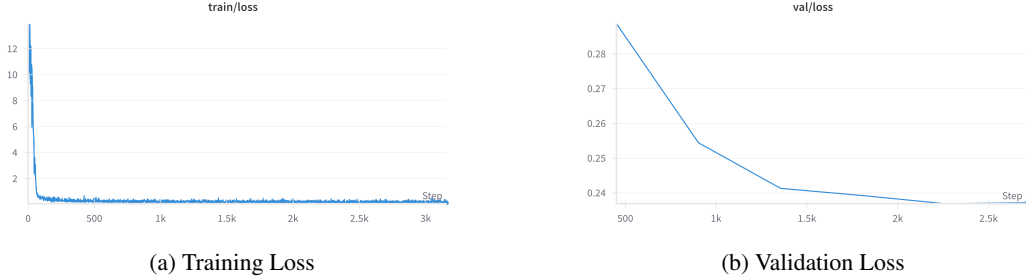


Figure 1: Loss curves for SFT training

RLOO For RLOO, we observed an unstable learning through its loss curve, as seen in Figure 2. When evaluated on a test set, the fine-tuned model performed worse than the SFT fine-tuned model, scoring 0.2385. We suspect the sparse rewards combined with a small batch size led to this high variance. Due to computational limitation, we couldn't launch training runs with more than 1 batch or $k=4$ generations per prompt, as we were running into out of memory errors.

Extension Interestingly, we observe a relative consistency that removing a specific subset of characters from the prompt can significantly boost performance of the model, and furthermore that relatively small changes to the prompt can lead to better performance. For example, in prompt format 2, the only change that is made to the prompt is removing the tokens: `for example <answer> (1 + 2) / 3 </answer>`. However, this boosts model performance significantly for almost all values of n , indicating that at least at inference time, this line from the prompt is distracting and unnecessary. Even more dramatically, with prompt format 4, we have removed nearly 25% of the total string. Yet even in this scenario, the model performs similarly or slightly better than with the original prompt. We believe this to be a nice illustration of how modern english is a relatively inefficient language, and often includes filler words inessential to relaying the intended message.

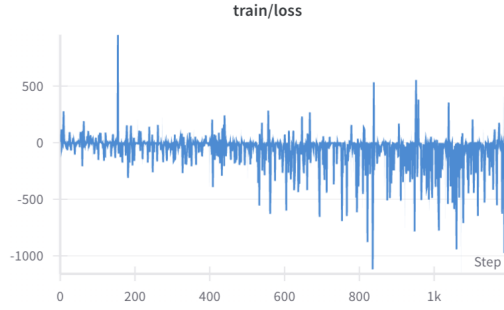


Figure 2: Loss curve for RLOO

In addition to raw performance, we also compare compute time during evaluation. That is, in Table 2, we record the total compute time to generate $31 = 1 + 2 + 4 + 8 + 16$ sample completions per prompt in the provided test dataset. Remarkably, we notice that for prompt format 4, which sacrifices essentially no performance, we see over a 10% speedup in compute time. We believe this to simply be because reducing the number of tokens allows the model to process prompts slightly faster, which predictably yields faster inference compute times.

We found it interesting to investigate the variance of the generations of the model to confirm that the model was behaving as expected. These results are presented in Figure 4. For this figure, variance is computed as follows. For each prompt in the dataset, we sample n completions from the model. We compute the score for each of these completions using the provided reward function for Countdown. For each prompt, we choose the best score, and compute the variance of the best scores over all the prompts in the dataset. As shown in the figure, we observe that for the original prompt format and prompt format 5 we see the expected drop in variance as we increase n . On the other hand, for prompt format 7, we see the opposite trend, with the variance starting low and increasing. This can be explained by the fact that we expect the scores for this prompt format to all be centered around 0 (they should be low because most of the prompt has been removed). We then observe the variance to increase since more samples from the distribution allow the model to succeed by random luck.

We also note that prompt format 3 is a notable exception to most of these trends. We believe this to be because, as stated more, in prompt format 3 we remove critical tokens from the prompt, which causes model generations to sharply decrease in quality. We believe this to be an important result because it emphasizes the importance of identifying which pieces of the prompt are of critical importance to the model and which others can be safely removed. Intuitively, it is not obvious that removing the line `Show your work in <think> </think>` tags should lead to a significant reduction of performance in the model, but upon further analysis it becomes clear why this is the case. This particular case is explored further in Section 5.2. To quote Einstein, "Everything should be made as simple as possible, but not simpler".

Finally, after retraining the base Qwen 2.5 0.5B model with SFT loss by preprocessing the Warmstart dataset to change queries to prompt format 4, we found that the performance degraded to a score of 0.2460 from the baseline of 0.3355. Prompt format omits the example in the query, which likely plays a critical role in teaching the model the expected output of a properly formatted equation. Subtle changes like changing `"and return the final answer..."` to `"return answer..."` might reduce LLM interpretability especially if the base model was trained with grammatically correct and full-sentence data.

	OG	F2	F3*	F4	F5	F6	F7
n=1	0.292	0.370	0.012	0.311	0.314	0.202	0.088
n=2	0.460	0.483	0.048	0.473	0.478	0.399	0.168
n=4	0.638	0.671	0.090	0.658	0.603	0.535	0.319
n=8	0.775	0.766	0.156	0.712	0.721	0.667	0.467
n=16	0.824	0.833	0.217	0.842	0.793	0.806	0.590
Cmp time (s)	306	299	285	271	270	266	247

Table 2: Average scores for each prompt format versus the number of completions sampled. Compute time is total compute time summed over all values of n

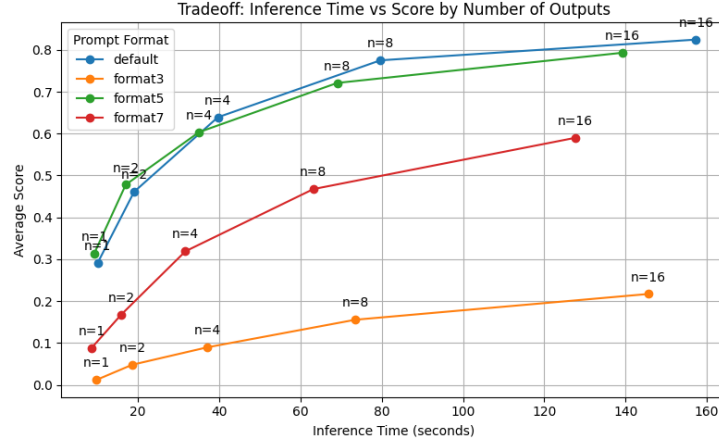


Figure 3: Plotting the accuracy of the model for a variety of n, d pairs

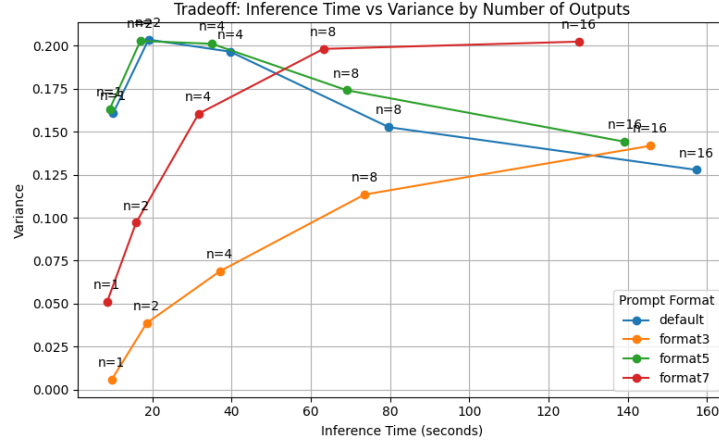


Figure 4: Plotting the accuracy of the model for a variety of n, d pairs

5.2 Qualitative Analysis

For qualitative evaluation, we found it interesting to look into why prompt format 3 performed so much worse than the other formats. From observing several sample completions returned by the model, it became clear that the difference in performance was a result of two underlying causes. First, the model struggled to return results in the correct form between answer tags. This may have been because neglecting to include `<think>` tags in the prompt made the model less likely to attend to the `<answer>` tags as well.

Secondly, we noticed that neglecting this line about including the thought process between think tags made the model stop thinking as much as it normally would. Two sample completions exemplifying this are shown in Appendix B. Note that with prompt format 2, the model performs more extensive and organized reasoning than with prompt format 3. Furthermore, note that with prompt format 2, the answer is returned correctly between <answer> tags whereas in prompt format 3 these are entirely missing.

The consequences of using the modified prompt structure (prompt format 4 specifically) to train the base model with SFT are also demonstrated in Appendix B with example generations. When fine-tuned with prompts where some of the essential words were removed, the model fails to converge on a solution and iterate through many incorrect subtractions. Removing the output example and weakening instruction phrasing could have led to this result. We hypothesize that the reason this did not hurt performance during test-time of the baseline SFT model was because it already understands the prompt structure and tags. This implies that prompts with examples and extra demonstrations are critical for guiding output format during early fine-tuning or when formatting errors are common.

6 Discussion and Future Work

Throughout this paper, we show that in the context of the Countdown task, removing unnecessary tokens from the prompt can be an effective way to reduce compute time during inference and even improve performance. However, there are a number of limitations to our method worth mentioning. First, tokens in the default prompt were categorized into critical and non-critical manually, which will often be impractical. We believe that it would be interesting for future research to train a model to classify tokens in a prompt into critical vs non-critical and use that information to cut out filler tokens from prompts. This would be a simple and scalable way to reduce compute times during inference.

Secondly, we note that the scale of our work here is limited to the Countdown task. While we believe that it is likely that the methods mentioned in this paper generalize to other mathematical reasoning tasks, at this time it is unclear to what extent these methods improve other domains. In particular, we would find it interesting to explore whether this method of prompt engineering can be generalized to broader conversation tasks, and find this to be an exciting place for future work.

7 Conclusion

In this work, we present a method to fine-tune a large language model, first with supervised fine tuning and then with RLOO, to improve performance on the Countdown task. We then demonstrate how prompt engineering and output sampling can be used to improve mathematical reasoning. In particular, we find that reducing the length of the prompt by removing non-critical tokens can both boost model performance and reduce compute costs during test-time inference. We believe this to be a novel and interesting result since it demonstrates that in the context of mathematical reasoning, decreasing prompt lengths by simply cutting filler tokens after a user has inputted a prompt may be able to both improve generation quality and decrease compute times, and since this phenomenon has been underexplored in prior literature.

8 Team Contributions

- **Nomin-Erdene Bayarsaikhan:** Set up the dataloader and training loop for SFT as well as best-N-sampling method.
- **Ronny Junkins:** Implemented the RLOO method as well as the prompt engineering analysis.

Changes from Proposal In our proposal, we planned to implement DPO and its variant but due to time and compute limitation, we mainly focused on RLOO and test time inference experiments.

References

Zhenni Bi, Kai Han, Chuanjian Liu, Yehui Tang, and Yunhe Wang. 2025. Forest-of-Thought: Scaling Test-Time Compute for Enhancing LLM Reasoning. arXiv:2412.09078v5 [cs.CL]

<https://arxiv.org/abs/2412.09078v5>

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 [cs.CL] <https://arxiv.org/pdf/2110.14168>

Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. 2025. Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs. arXiv:2503.01307 [cs.CL] <https://arxiv.org/abs/2503.01307>

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798 [cs.CL] <https://arxiv.org/pdf/2310.01798>

Aldo Pareja, Nikhil Shivakumar Nayak, Hao Wang, Krishnateja Killamsetty, Shivchander Sudalairaj, Wenlong Zhao, Seungwook Han, Abhishek Bhandwalidar, Guangxuan Xu, Kai Xu, Ligong Han, Luke Inglis, and Akash Srivastava. 2024. Unveiling the Secret Recipe: A Guide For Supervised Fine-Tuning Small LLMs. arXiv:2412.13337 [cs.CL] <https://arxiv.org/abs/2412.13337>

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. arXiv:2408.03314 [cs.CL] <https://arxiv.org/abs/2408.03314>

Kaya Stechly, Matthew Marquez, and Subbarao Kambhampati. 2023. GPT-4 Doesn't Know It's Wrong: An Analysis of Iterative Prompting for Reasoning Problems. arXiv:2310.12397 [cs.CL] <https://arxiv.org/pdf/2310.12397>

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 [cs.CL] <https://arxiv.org/pdf/2201.11903>

Hattie Zhou, Azade Nova, Hugo Larochelle, Aaron Courville, Behnam Neyshabur, and Hanie Sedghi. 2022. Teaching Algorithmic Reasoning via In-context Learning. arXiv:2211.09066 [cs.CL] <https://arxiv.org/pdf/2211.09066>

A All Prompt Formats

Default Prompt Format:

"A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer.\n User: Using the numbers numbers, create an equation that equals target. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>.\n Assistant: Let me solve this step by step."

Prompt Format 2:

"A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer.\n User: Using the numbers numbers, create an equation that equals target. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags.\n Assistant: Let me solve this step by step."

Prompt Format 3:

"A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer.\n User: Using the numbers numbers, create an equation that equals target. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Return the final answer in <answer> </answer> tags.\n Assistant: Let me solve this step by step."

Prompt Format 4:

"A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant thinks about the reasoning process then provides the user with the answer.\n User: Using the numbers numbers, create an equation that equals target. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. return answer in <answer> </answer> tags.\n Assistant: Let me solve this step by step."

Prompt Format 5:

"A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant thinks then provides the user with the answer.\n User: Using the numbers numbers, create an equation that equals target. You can use operations (+, -, *, /) and each number only once. Show your work in <think> </think> tags. return answer in <answer> </answer> tags.\n Assistant: Let me solve this step by step."

Prompt Format 6:

"A conversation between User and Assistant. user asks a question, the Assistant solves it. assistant thinks then provides the answer.\n User: Using numbers numbers, create an equation that equals target. use operations (+, -, *, /) and each number once. Show your work in <think> </think> tags. return answer in <answer> </answer> tags.\n Assistant: Let me solve this step by step."

Prompt Format 7:

"conversation User and Assistant. user asks question, Assistant solves assistant provides the answer.\n User: Using numbers, create equation equals target. use (+, -, *, /) and each number once. Show work in <think> </think> return answer in <answer> </answer> tags.\n Assistant: Let me solve this"

B Qualitative Analysis Completions

===== Prompt Format 2 Sample Completion =====

TARGET: 45

NUMBERS: [39, 3, 3]

PROMPT: A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer.

User: Using the numbers [39, 3, 3], create an equation that equals 45. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags.

Assistant: Let me solve this step by step.

RAW OUTPUT: Since 45 is a prime number, I need to find ways to get there using the given numbers.

First, let's try to get close to 45 with basic operations:

$39 + 3 = 42$ (close but not quite)

$39 - 3 = 36$ (closer but not enough)

$39 * 3 = 137$ (too large)

$39 / 3 = 13$ (not helpful)

Let's work backwards from 45:

45 could be reached by adding 6 to 39

Or by subtracting 3 from 42

Or by adding 1 to 36

Looking at the numbers:

39 is closest to 45

We need to add 6 to 39

Let's try to find the way to get 6:

3 is the key intermediate step

$39 - 3 = 36$

$36 + 1 = 37$

$37 + 3 = 40$ (not quite)

Let's try a different approach:

$39 - 3 = 36$

$36 + 3 = 49$

$49 - 1 = 48$ (getting closer but still not right)

New approach:

$39 + 3 = 42$

$42 + 1 = 43$

$3 + 1 = 44$ (not quite)

Final attempt:

$39 + 3 = 42$

$42 + (3 - 1) = 42 + 2 = 44$

$44 + 1 = 45$ (This works!)

So we can get to 45 by:

1. First adding 3 to 39

2. Then adding 1 to the result of 39

3. Finally subtracting 1 from 42

</think>

<answer> $(39 + 3) + (3 - 1)$ </answer>

EXTRACTED ANSWER: $(39 + 3) + (3 - 1)$

SCORE: 0.1000

=====|

===== Prompt Format 3 Sample Completion =====

TARGET: 45

NUMBERS: [39, 3, 3]

PROMPT: A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer.

User: Using the numbers [39, 3, 3], create an equation that equals 45. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Return the final answer in <answer> </answer> tags.

Assistant: Let me solve this step by step.

RAW OUTPUT: Since 45 is relatively large, we'll need to use division to get smaller numbers.

First, let's try to get closer to 45 by reducing the large numbers.

39 is already close to 45, so we need to add 6 to it.

Looking at the other numbers: 3, 39, and 39

If we divide 39 by 3, we get 13

Then adding 3 to 13 would give us 40, which is close to 45

Let's try another approach:

What if we first divide 39 by 3 to get 13

Then add 3 to get 16

Then add 39 to get 55

Not quite right

Let's try another way:

$39 - 3 = 36$

$36 + 3 = 39$

$39 + 3 = 42$

Still not right

One more attempt:

$39 + 3 = 42$
 $42 + 3 = 45$
 This works! We can verify:
 First: $39 + 3 = 42$
 Then: $42 + 3 = 45$
 </answer>
 EXTRACTED ANSWER: None
 SCORE: 0.0000

===== Sample Completion from baseline (N=1 sampling and default prompt with the original dataset) =====
 TARGET: 21
 NUMBERS: [15, 5, 75, 90]
 PROMPT: A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer.
 User: Using the numbers [15, 5, 75, 90], create an equation that equals 21. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>.
 Assistant: Let me solve this step by step.
 RAW OUTPUT: Since 21 is relatively small compared to the given numbers, we need to use division to get down to 21.
 Let's try to find factors of 21: 1, 3, 7, 21
 Or numbers close to 21: 15, 21, 23, 24

Let's work backwards from 21:
 - 21 is $3 * 7$
 - 21 is also close to 15, so we might want to use 15 first
 - We need to get 6 more somehow using 5, 75, and 90

Let's try to get 6:
 $75 - 90 = -15$
 $90 - 75 = 15$
 $15 + 5 = 20$
 $20 + 1 = 21$

So if we can get 1 first, then add 15, we'll reach 21.
 $90 - 75 = 15$
 $15 + 5 = 20$
 $20 + 1 = 21$

This works! Let's verify:
 1. First subtract 75 from 90 to get 15
 2. Then add 5 to get 20
 3. Finally add 1 to get 21

The sequence would be:
 $(90 - 75) + 5 + 15 = 21$
 </think>
 <answer> $(90 - 75) + 5 + 15$ </answer>
 EXTRACTED ANSWER: $(90 - 75) + 5 + 15$
 SCORE: 0.1000

===== Sample Completion from finetuning with modified prompt structure (N=1 sampling and prompt format 4 used during training) =====
 TARGET: 21
 NUMBERS: [15, 5, 75, 90]
 PROMPT: A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant thinks about the reasoning process

then provides the user with the answer.

User: Using the numbers [15, 5, 75, 90], create an equation that equals 21.
You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. return answer in <answer> </answer> tags.

Assistant: Let me solve this step by step.

RAW OUTPUT: First, let's try to get close to 21 by looking at factors or multiples of 21.

$$21 = 3 * 7$$

$$21 = 3 * 7$$

Looking at the numbers:

15 is close to 21, we need to add 6 to it

5 is also close to 21, we need to add 16 to it

75 and 90 are much larger than 21

Let's try working with 75 and 90 first since they're larger:

$$90 - 75 = 15$$

$$15 * 5 = 75$$

$$75 - 15 = 60$$

$$60 - 90 = -30$$

Let's try a different approach:

$$75 - 5 = 70$$

$$70 - 15 = 55$$

$$55 - 90 = -35$$

One more attempt:

$$90 - 75 = 15$$

$$15 * 5 = 75$$

$$75 - 15 = 60$$

$$60 - 90 = -30$$

Let's try:

$$75 - 5 = 70$$

$$70 - 15 = 55$$

$$55 - 90 = -35$$

Final attempt:

$$90 - 75 = 15$$

$$15 * 5 = 75$$

$$75 - 15 = 60$$

$$60 - 90 = -30$$

I need to backtrack and try a different sequence.

Let's try:

$$75 - 5 = 70$$

$$70 - 15 = 55$$

$$55 - 90 = -35$$

One last try:

$$90 - 75 = 15$$

$$15 * 5 = 75$$

$$75 - 15 = 60$$

$$60 - 90 = -30$$

Final attempt:

75 - 5 = 70
70 - 15 = 55
55 - 90 = -35

I need to backtrack and try a different sequence.

Let's try:
90 - 75 = 15
15 * 5 = 75
75 - 15 = 60
60 - 90 = -30

Final attempt:
75 - 5 = 70
70 - 15 = 55
55 - 90 = -35

I need to backtrack and try a different sequence.

Let's try:
90 - 75 = 15
15 * 5 = 75
75 - 15 = 60
60 - 90 = -30

Final attempt:
75 - 5 = 70
70 - 15 = 55
55 - 90 = -35

I need to backtrack and try a different sequence.

Let's try:
90 - 75 = 15
15 * 5 = 75
75 - 15 = 60
60 - 90 = -30

Final attempt:
75 - 5 = 70
70 - 15 = 55
55 - 90 = -35

I need to backtrack and try a different sequence.

Let's try:
90 - 75 = 15
15 * 5 = 75
75 - 15 = 60
60 - 90 = -30

Final attempt:
75 - 5 = 70
70 - 15 = 55
55 - 90 = -35

I need to backtrack and try a different sequence.

Let's try:

90 - 75 = 15
15 * 5 = 75
75 - 15 = 60
60 - 90 = -30

Final attempt:
75 - 5 = 70
70 - 15 = 55
55 - 90 = -35

I need to backtrack and try a different sequence.

Let's try:
90 - 75 = 15
15 * 5 = 75
75 - 15 = 60
60 - 90 = -30

Final attempt:
75 - 5 = 70
70 - 15 = 55
55 - 90 = -35

I need to backtrack and try a different sequence.

Let's try:
90 -
EXTRACTED ANSWER: None